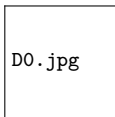


# Lecture 18: Transactions & Reliability

## ACID, Concurrency Control, Deadlocks, ARIES & RAID

Z2004: Database Management Systems



IIT Madras Zanzibar

Friday, 10 April 2026

## Preventing Chaos in a Concurrent World

Until now, we assumed a single user running a single query on a database that never loses power. Today, we face reality: thousands of users hit databases simultaneously, and hardware fails constantly.

### Outline:

- ➊ **Transactions:** The logical unit of work.
- ➋ **The ACID Properties:** The DBMS contract of safety.
- ➌ **Concurrency Control:** Schedules, Locks, and Two-Phase Locking (2PL).
- ➍ **Deadlocks:** Detection and resolution in standoffs.
- ➎ **Crash Recovery:** The Buffer Pool problem and Write-Ahead Logging (WAL).
- ➏ **The ARIES Algorithm:** Analysis, Redo, and Undo.
- ➐ **Hardware Reliability:** RAID architectures.

## Traffic Jams and Tripping Hazards

### Problem 1: Concurrency

- ▶ Taylor Swift tickets go on sale.
- ▶ User A and User B click "Buy" on Seat 1A at the exact same millisecond.
- ▶ Both read the seat as "Available".
- ▶ Both write "Sold".
- ▶ **Result:** Double-booking disaster.

### Problem 2: System Failures

- ▶ You send \$100 to Mary on MPESA.
- ▶ `UPDATE Accounts SET bal = bal - 100 WHERE name = Tom;`
- ▶ ⚡ **POWER OUTAGE!** Server dies before updating Mary's account.
- ▶ **Result:** \$100 vanishes from the economy.

# What is a Transaction?

## The Logical Unit of Work

A **Transaction (Tx)** is a sequence of one or more SQL operations treated as a **single, indivisible wrapper**. It must entirely succeed or entirely fail.

### The SQL Commands:

- ▶ **BEGIN:** Starts the safety wrapper.
- ▶ **COMMIT:** Saves all changes permanently. Cannot be undone.
- ▶ **ROLLBACK:** Aborts the transaction. Reverts the database to the exact state before **BEGIN**.

```
BEGIN TRANSACTION;  
    -- Deduct from Alice  
    UPDATE Accounts  
    SET Bal = Bal - 50 WHERE Name = 'Alice';  
  
    -- Add to Bob  
    UPDATE Accounts  
    SET Bal = Bal + 50 WHERE Name = 'Bob';  
COMMIT;
```

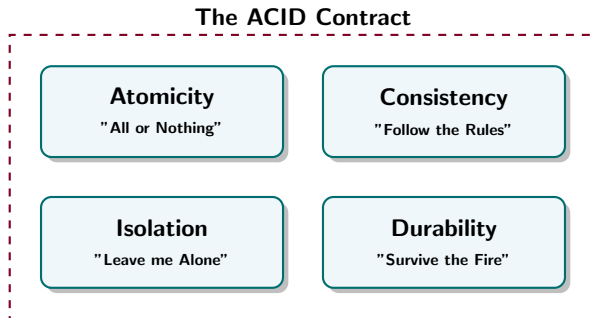
# Transaction

transaction.png

# The ACID Properties

## The Database Guarantee

To ensure data survives the chaos of concurrency and crashes, every relational DBMS guarantees four properties for every transaction.



## Atomicity

*"All or Nothing."*

- ▶ A transaction cannot be left half-finished.
- ▶ If step 3 of 5 fails, the DBMS automatically executes a ROLLBACK for steps 1 and 2.
- ▶ Solves the Venmo disappearing money problem.

## Consistency

*"Follow the Rules."*

- ▶ The database must transition from one valid state to another.
- ▶ If an UPDATE results in a negative balance, and the column has a CHECK ( $bal \geq 0$ ) constraint, the transaction is rejected.
- ▶ Ensures Primary Keys, Foreign Keys, and Triggers are honored.

## Isolation

*"Leave me Alone."*

- ▶ Transactions executing concurrently must appear as if they executed sequentially (one after the other).
- ▶ User B cannot see User A's uncommitted data.
- ▶ Prevents the Taylor Swift double-booking problem.

## Durability

*"Survive the Fire."*

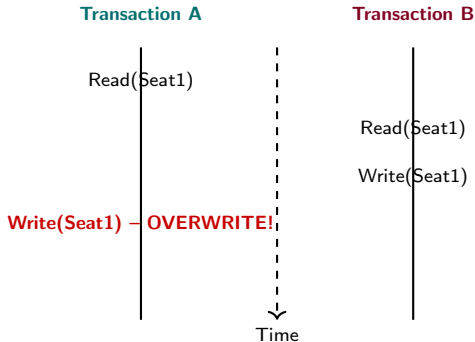
- ▶ Once a transaction issues a COMMIT, the changes are permanent.
- ▶ Even if someone trips over the server power cord 1 millisecond later, the data must still be there when the server reboots.



# Deep Dive: Isolation & Schedules

## Interleaving Operations

If we run transactions strictly one after another (Serial Schedule), it's safe, but terribly slow. To maximize CPU usage, the DBMS *interleaves* operations from multiple transactions.



**Goal:** The DBMS must guarantee **Serializability** (the interleaved schedule produces the exact same result as a serial schedule).

## The Bathroom Door Analogy

To enforce Isolation, the DBMS uses **Locks**. Before reading or writing a row, a transaction must acquire a lock on that row.

### Shared Lock (S-Lock)

Used for SELECT.

- ▶ Multiple transactions can hold an S-Lock on the same row simultaneously.
- ▶ "We can all look at the poster at the same time."

### Exclusive Lock (X-Lock)

Used for UPDATE, INSERT, DELETE.

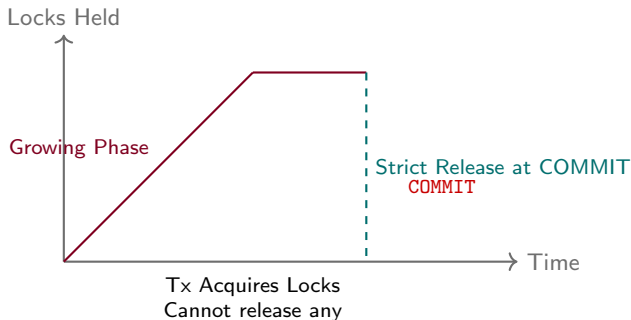
- ▶ Only **one** transaction can hold an X-Lock.
- ▶ If Tx A holds an X-Lock, Tx B cannot even get an S-Lock. Tx B is blocked and must wait!

lock.png

# Two-Phase Locking (2PL)

## The Golden Rule of Serializability

Just using locks isn't enough; you must acquire and release them in a specific pattern to guarantee safety. The industry standard is **Strict Two-Phase Locking (2PL)**.



**Rule:** A transaction cannot acquire ANY new locks once it releases its first lock. In Strict 2PL, all locks are held until the very end (COMMIT/ROLLBACK).

## 2PL: what it really means

### Two phases for each transaction (Tx):

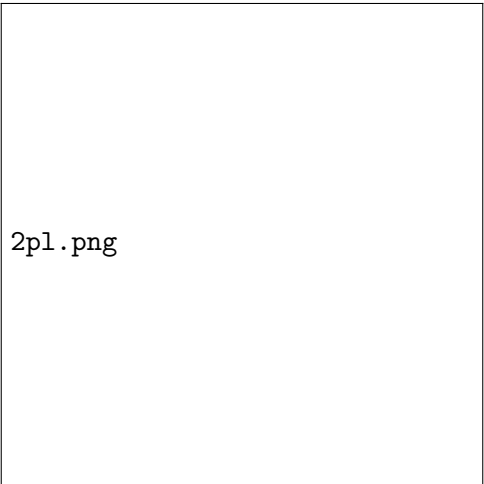
- ▶ **Growing phase:** Tx can **acquire** locks (S/X), but **releases none**.
- ▶ **Shrinking phase:** Tx can **release** locks, but **acquires no new ones**.

### Strict 2PL (what DBs typically implement):

- ▶ Hold **all X-locks** (often all locks) until COMMIT/ROLLBACK.
- ▶ Prevents **dirty reads** and guarantees **conflict-serializability**.

Memory hook: **Lock, lock, lock... then unlock at the end.**

## 2PL Locking Protocol



2pl.png

# Strict 2PL: Timeline example (who waits for whom)

**Data items:** rows A and B (X = exclusive lock)

| Time | Tx1 (transfer: A $\rightarrow$ B) | Tx2 (reads A)                          |
|------|-----------------------------------|----------------------------------------|
| 1    | X(A); write A                     |                                        |
| 2    | X(B); write B                     |                                        |
| 3    | (still holding X(A), X(B))        | request S(A) $\rightarrow$ <b>WAIT</b> |
| 4    | COMMIT; release all locks         | S(A) granted; read A                   |

**Key idea:** Tx2 is forced to see Tx1's **committed** result (no dirty read), because Tx1 doesn't release locks early.

# The Deadlock Problem

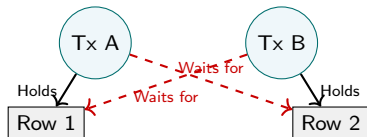
## The Unbreakable Standoff

Because 2PL forces transactions to hold locks and wait for others, it introduces a fatal flaw: **Deadlock**.

**Definition:** A deadlock happens when a set of transactions are **each waiting for a lock held by another** in the set, so **none of them can make progress**.

### The Scenario:

- 1 Tx A locks Row 1 (X-Lock).
- 2 Tx B locks Row 2 (X-Lock).
- 3 Tx A requests Row 2. It blocks and waits for Tx B.
- 4 Tx B requests Row 1. It blocks and waits for Tx A.



**Result:** They will wait forever. The system freezes.



# Deadlock Detection & Resolution

## Breaking the Cycle

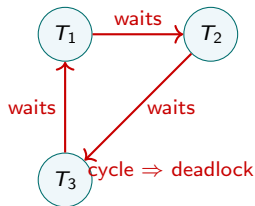
The DBMS runs a background program (the **Deadlock Detector**) periodically.

**Step-by-step (think: DS cycle detection):**

- 1 **Build a wait-for graph (WFG)**: node = transaction.
- 2 Add edge  $T_i \rightarrow T_j$  if  $T_i$  is **blocked** waiting for a lock held by  $T_j$ .
- 3 **Detect a cycle** in the directed graph (DFS colors / back-edge).  
**Cycle**  $\Rightarrow$  deadlock.
- 4 **Pick a victim** (min cost): youngest Tx, fewest rows updated, least log written, etc.
- 5 **Rollback victim** to release locks; others continue; victim may restart.

**Quick note:** Some systems also use **timeouts** ("wait too long"  $\Rightarrow$  abort) as a simpler fallback.

**Tiny WFG example**



# Crash Recovery: The Buffer Pool Problem

## Why Crashes cause Data Corruption

To understand Durability, we must remember Lecture 16: Databases manipulate data in RAM (Buffer Pool), not directly on the Disk. Disk writes are slow.

### STEAL Policy (Undo Problem)

- ▶ The DB writes uncommitted data to disk to free up RAM.
- ▶ If a crash happens, disk contains garbage data from a failed transaction. We need to **UNDO** it.

### NO-FORCE Policy (Redo Problem)

- ▶ A Tx commits in RAM, but the DB delays writing the data to the disk to save time.
- ▶ If a crash happens, committed data is lost! We need to **REDO** it.

**The DBMS Dilemma:** How do we undo bad data and redo lost data after a power outage wipes the RAM?

# The Write-Ahead Log (WAL)

## The Database Receipt System

The solution is the **Write-Ahead Log (WAL)**. It is an append-only file on the hard drive that records *intentions* before data pages are actually modified.

Log File  
(Disk)

LSN 101: Tx1 UPDATE Account 'A' (Old: 100, New: 50)

LSN 102: Tx1 UPDATE Account 'B' (Old: 50, New: 100)

LSN 103: Tx1 COMMIT

LSN 104: Tx2 UPDATE Account 'C' (Old: 20, New: 0)

### The WAL Rules:

- ▶ Rule 1: A log record containing "Old Data" must be written to disk **before** the actual dirty page is written to disk (enables UNDO).
- ▶ Rule 2: A transaction cannot return "Success" to the user until its COMMIT log record is forced to disk (enables REDO).

# ARIES Crash Recovery Algorithm

## Algorithms for Recovery and Isolation Exploiting Semantics

Invented at IBM, ARIES is the industry-standard algorithm used by PostgreSQL and MySQL to recover from a crash using the WAL. When the server reboots, ARIES runs in 3 phases.

- ➊ **Analysis Phase:** Reads the WAL forward to figure out exactly which transactions were active at the moment of the crash, and which pages might be corrupted.
- ➋ **Redo Phase:** Reads the WAL forward. It **repeats history**, restoring the database to the exact state it was in at the millisecond of the crash (including recreating the uncommitted garbage!).
- ➌ **Undo Phase:** Reads the WAL backwards. It takes the active transactions identified in Phase 1 and surgically removes their changes using the "Old Data" from the logs.

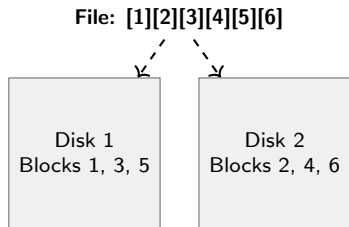
## What if the Hard Drive Dies?

WAL protects us from power outages because the logs are safe on the disk. But what if the physical SSD catches fire and turns to ash? No software algorithm can recover melted silicon.

- ▶ **Database Backups:** Taking a complete snapshot of the DB every 24 hours. (Provides disaster recovery, but you lose the last 24 hours of data).
- ▶ **High Availability (HA):** Replicating the database over the network to a secondary server in a different city.
- ▶ **RAID (Redundant Array of Independent Disks):** Combining multiple physical hard drives into one logical unit inside the server to survive a drive failure seamlessly.

# RAID 0 (Striping)

## Built for Speed, Not Safety

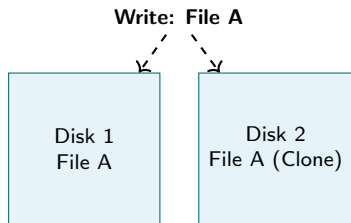


Data is split evenly across two or more drives.

- ▶ **Pros:** Double the read/write speed. You write half the file to Disk 1 and half to Disk 2 simultaneously.
- ▶ **Cons:** Zero redundancy. If Disk 1 dies, you lose half your blocks, rendering the entire database corrupt.
- ▶ **Verdict:** Never use RAID 0 for a database!

# RAID 1 (Mirroring)

## The Ultimate Clone



Every block of data written to Disk 1 is simultaneously cloned to Disk 2.

- ▶ **Pros:** If Disk 1 dies, the system switches to Disk 2 instantly. Zero downtime. Zero data loss.
- ▶ **Cons:** Very expensive. You buy 4TB of hard drives, but only get 2TB of usable space (50% efficiency).

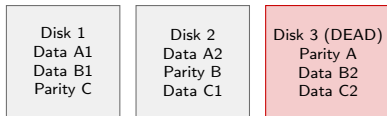
# RAID 5 (Block-Level Striping with Parity)

## The Industry Standard Compromise

Requires at least 3 disks. It stripes data across the drives, but reserves space for **parity**.

**Parity (definition):** a bitwise **XOR (Exclusive OR)** of the data blocks in the stripe; if any *one* disk is missing, XOR lets us **reconstruct the missing block**.

**Parity-based RAID family:** RAID 3, RAID 4, RAID 5, and RAID 6.



### How Parity Works (XOR Logic):

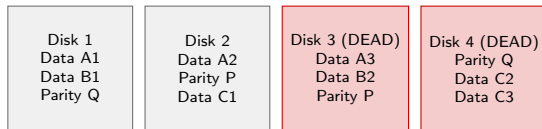
- ▶  $A1 + A2 = \text{ParityA}$
- ▶ If Disk 3 dies, we lose *ParityA*, *DataB2*, and *DataC2*.
- ▶ The server can calculate *DataB2* instantly by subtracting *DataB1* from *ParityB*!
- ▶ We replace the dead drive, and the math rebuilds it automatically.



# RAID 6 (Striping with Double Parity)

## Like RAID 5, but survives two disk failures

RAID 6 stripes data and stores **two independent parity blocks** per stripe (often called **P** and **Q**).



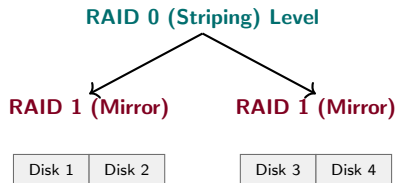
**Key guarantee:** can lose **any 2 disks** and still rebuild.

### How it works (high level):

- ▶ Two parity equations are stored per stripe: **P** (XOR-like) and **Q** (a second, independent parity).
- ▶ With **one** missing disk: rebuild like RAID 5.
- ▶ With **two** missing disks: the DBMS/RAID controller uses **P and Q together** to solve for both missing blocks.
- ▶ **Tradeoff:** higher write cost than RAID 5 (must update two parities).

**When to use:** large arrays where a 2nd disk failure during rebuild is realistic.

## The Enterprise Database Choice



Combines the speed of RAID 0 with the safety of RAID 1. Requires 4+ disks.

- ▶ Data is striped across two separate RAID 1 mirror arrays.
- ▶ **Pros:** Incredible database performance, and it can survive multiple drive failures (as long as they aren't in the same mirror pair).
- ▶ **Cons:** Most expensive setup (50% usable space).

## Key Takeaways

- ✓ **ACID:** Atomicity (All/Nothing), Consistency (Rules), Isolation (Concurrency), Durability (Crashes).
- ✓ **2PL:** Strict Two-Phase locking ensures safe concurrent schedules, but introduces the risk of Deadlocks.
- ✓ **WAL:** The Write-Ahead Log acts as the source of truth if RAM is lost during a power failure.
- ✓ **ARIES:** Analyzes the log, Redoes history to recreate the exact crash state, then Undoes uncommitted transactions.
- ✓ **RAID:** RAID 5 and RAID 10 provide hardware redundancy so databases can survive physical drive failure without downtime.

## To reinforce today's ideas

- ▶ Review the lecture slides: **Transactions & Reliability (L18)**.
- ▶ Course textbook: read the sections on
  - ◇ **Transactions and ACID**
  - ◇ **Concurrency control** (2PL, isolation levels, deadlocks)
  - ◇ **Recovery** (WAL / ARIES / checkpoints)
  - ◇ **Storage reliability** (RAID parity and mirroring)
- ▶ Practice: write 2–3 interleavings and label the anomaly (dirty read / lost update / deadlock).